

Manuale Tecnico: CineMax

Università degli studi dell'Insubria: [Progetto interdisciplinare A 2026](#)

Autori: [Omar Hajjoubi](#), [Filippo Mare](#), [Glorio Martignoni](#), [Simone Pica](#)

INDICE

- 1. Introduzione**
- 2. Struttura dell'applicazione**
- 3. Scelte architettureali**
- 4. Strutture dati**
- 5. Gestione file**

INTRODUZIONE

CineMax è un progetto sviluppato per il corso di laboratorio interdisciplinare A della facoltà di informatica dell'Università degli Studi dell'Insubria.

Il progetto utilizza la versione di java 26.0.1.

STRUTTURA DELL'APPLICAZIONE

A livello architetturale è possibile dividere l'applicazione in 3 livelli:

- 1. Interfaccia Utente (CLI):** Tramite terminale che si avvia utilizzando il file avvio.bat per Windows oppure il file avvio.command per macOS o Linux;
- 2. Logica di Applicazione:** creata e gestita attraverso la classe "GestioneCinema" ;
- 3. Gestione di file:** attraverso file di tipo byte, che consentono la serializzazione degli oggetti.

SCELTE ARCHITETTURALI

Il programma è suddiviso nelle seguenti classi:

1. Utente

La classe gestisce le informazioni degli utenti e il modo in cui queste vengono settate e restituite ai metodi chiamanti.

Gli utenti possono essere:

- a. Cliente
- b. Bigliettaio
- c. Proiezionista

Per questo motivo utilizziamo una gerarchia che tramite l'utilizzo delle enum a, b, c che consentono di differenziare gli utenti.

2. Ruolo

Enum che permette di suddividere gli utenti in base al ruolo che ricoprono.

3. Proiezione

La classe gestisce le informazioni delle proiezioni, il modo in cui queste vengono settate e restituite ai metodi chiamanti. Inoltre consente l'aggiunta, modifica o cancellazione di una proiezione.

4. Prenotazione

La classe gestisce le informazioni delle prenotazioni, il modo in cui queste vengono settate e restituite ai metodi chiamanti. Inoltre consente l'aggiunta, modifica o cancellazione di una prenotazione.

5. Film

La classe gestisce le informazioni dei film, il modo in cui queste vengono settate e restituite ai metodi chiamanti.

6. Regista

La classe gestisce le informazioni dei film, il modo in cui queste vengono settate e restituite ai metodi chiamanti.

7. Genere

Enum che permette di gestire i diversi generi di un determinato film.

8. Data

La classe serve a rappresentare la data di una proiezione e la data di nascita dei nostri utenti.

9. Ora

La classe rappresenta l'orario per le nostre proiezioni.

10. Luogo

La classe serve a rappresentare il luogo di nascita dei nostri utenti.

11. IllegalArgumentException

La classe rappresenta l'eccezione sollevata dal programma quando l'utente inserisce dati errati, inconsistenti o parziali.

12. IO

La classe gestisce tutti i metodi di lettura e scrittura per interagire con l'utente. In questo caso la lettura utilizza lo stream tastiera, mentre la scrittura lo schermo entrambi tramite CLI.

13. GestoreCinema

La classe gestisce tutte le possibili operazioni per il cinema e la scrittura sui file.

14. CineMax

La classe è il nostro "main", dove gestiamo i vari menù per i diversi utenti e le operazioni che ne conseguono.

STRUTTURE DATI

Le strutture dati utilizzate in questo progetto sono:

1. **Array di stringhe**
2. **LinkedList**
3. **File di bytes**

Gli array di stringhe sono stati usati principalmente nelle classi GestoreCinema e CineMax per gestire le stampe in riga di comando e gli input inseriti dall'utente.

Vantaggi:

- Efficienti e semplici: accesso diretto e semplice ai dati.

Svantaggi:

- Non abbiamo individuato svantaggi significativi nell'utilizzo degli array di stringhe.

Le LinkedList sono state utilizzate principalmente nelle classi GestoreCinema e CineMax per la gestione dei dati del programma. Le LinkedList contengono tutti i dati utilizzati durante l'esecuzione che successivamente, al termine dell'applicazione, verranno scritti sul file "archivio.dat".

Vantaggi:

- Efficienti e semplici: accesso diretto e semplice ai dati.

Svantaggi:

- Gestione casting: scrivendo sui file LinkedList di oggetti diversi tra loro, divisi da un token "*", può diventare articolata la gestione della lettura dei file. Per risolverlo abbiamo impostato una struttura del file molto robusta e l'utilizzo di: contatori, switch e costrutti di verifica della classe di origine (come instance of).

Il file di bytes è stato utilizzato come database per la nostra applicazione. La scelta dei file di tipo byte è dovuta all'utilizzo della serializzazione delle classi: Utente (e relativa gerarchia), Film, Regista, Proiezione e Prenotazione; in questo modo la scrittura e lettura delle LinkedList che il programma utilizza quando è in esecuzione per permettere la modifica del dataset è di facile e rapida gestione.

Vantaggi:

- Efficienti e semplici: accesso diretto e semplice ai dati.
- Gestione della lettura e scrittura sul file: immediata dovuta all'utilizzo della serializzazione.

Svantaggi:

- Leggibilità: utilizzando i file di byte con serializzazione la lettura del file richiede l'utilizzo del metodo letturaFile() oppure editor specifici.

GESTIONE FILE

I file utilizzati sono di tipo byte, il nostro programma all'avvio riempie dal file "archivio.dat" le LinkedList di GestoreCinema, le quali per tutta l'esecuzione del programma salvano le interazioni con gli utenti (aggiunte, modifiche o cancellazioni varie), infine al termine del programma le stesse vengono scritte nel file (divise da un token "**") tramite l'utilizzo della serializzazione degli oggetti.

Nei file troviamo le seguenti liste:

- ContatoreSezioni
- ListaUtenti
- ListaFilm
- ListaPrenotazioni
- ListaProiezioni
- ListaRegisti

Il file utilizza percorsi relativi e metodi di verifica di consistenza per garantire il funzionamento anche in condizioni di file: danneggiato, non trovato o inesistente.